
Getting Started on the Cluster Onboarding

These are rough instructions for getting started on the Yale YCRC servers. Accurate as of June 30 2026.

1 Requesting an Account

To start the process request an account on one or more of the Yale administered clusters. Currently Bouchet is the main one. You can find the account request form here: docs.ycrc.yale.edu/clusters-at-yale/

If you are an undergraduate or high school student your department or school, should be School of Engineering, or Department of Mechanical Engineering.

Dont forget to enter Corey O'Hern (corey.ohern@yale.edu) as your PI. He will need to approve your account.

The account approval process may take 1-2 business days or as soon as an hour depending on who is available to process the request. If you want to get a jump on things you can familiarize yourself with the tools listed below and possibly learn about how to use them (or tools like them) on your local computer.

2 Getting started on Bouchet

You can access bouchet most easily via the Open on Demand portal 'ood': ood-bouchet.ycrc.yale.edu/pun/sys/dashboard.

There you can open a command line shell by clicking "Bouchet Shell Access" on the top right.

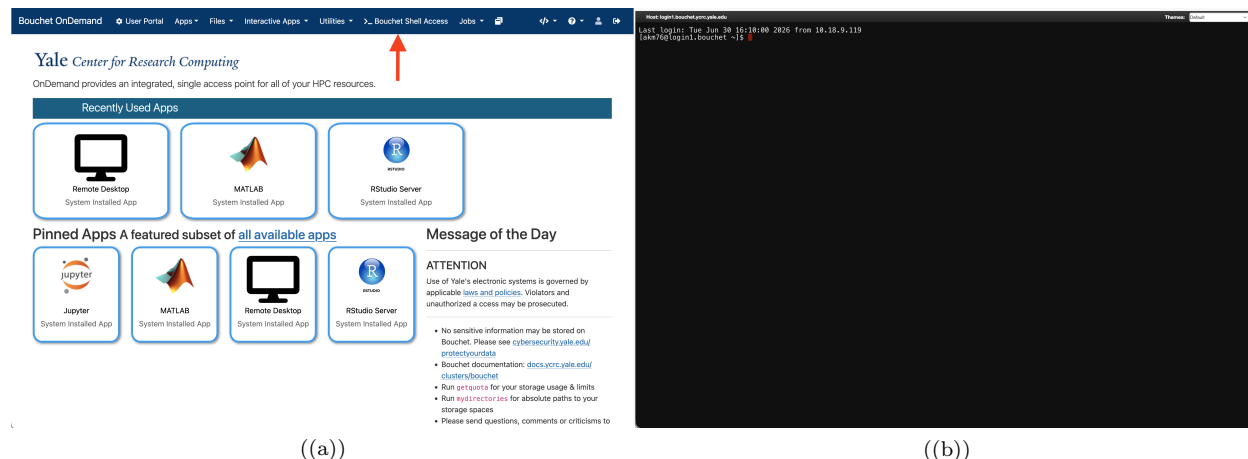


Figure 1: (a) OOD Portal. (b) New Shell.

Beginner linux user checklist, if you have a mac then you can try some of these on your own computer (be careful there are no undos for command line tools) if you're using a windows computer there are equivalent tools, but these tools will not work exactly the same way as the tools installed on the cluster.

1. Navigating a file system from the command line: `cd`, `ls`, `pwd`, `touch`, `cp`, `mv`, `rm`, `mkdir`
2. Command line editors: `vim` or `nano`
3. Remote shell and login via commandline: `ssh` (tutorial on how to set that up at YCRC)
4. Direct file transfer: `scp` or `rsync` (use the transfer nodes: docs.ycrc.yale.edu/data/transfer/)

You can familiarize yourself with command line tools on your own computer (look up how to navigate your computer on the command line and edit files with a command line editor like `vim` or `nano`).

If you have had little or no experience using a computer from the command line before take an hour or so and familiarize yourself with this. Try using the navigation tools mentioned above to move around in your home directory. Look up each of the command line tools and what they're use for.

Finally complete a `vim` or `nano` tutorial and write a sentence or two in a text file from the command line before moving on. Save it and open it to see how it looks.

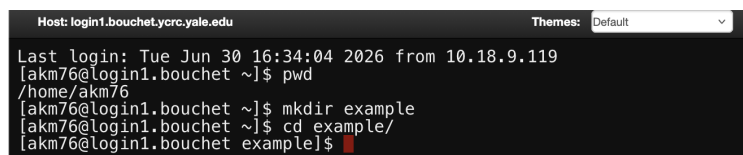
3 Running Jobs

When you use the cluster from shell you got from the ood portal, you are on what is called a ‘login’ node. These are not meant for computation or transferring files, just setting up jobs and checking on them, with some light editing here and there. If you want to run jobs (and don’t want the sysadmins to ask why you’re using the login nodes to run code) you’ll need to learn how to submit jobs to the cluster resource manager. For this we use a software called Slurm, which has its own syntax that I will touch on briefly.

To run a job on the cluster you might need to use some software like conda, or matlab that is not there by default. The yale clusters use the keyword ‘module’ more documentation about that can be found here: <https://docs.ycrc.yale.edu/applications/modules/>. In this example I will use matlab and a software called dSQ.

A job on a cluster is like a single line command on the command line that you send to a computer, and it runs it and gives you back any output files created from it. You will not be able to see any plots unless you explicitly save them. They usually save files by default in the directory from which the job is submitted (the current directory (where you ‘are’) in the terminal when you submit the job). It is therefore convient to create a directory for a job or a project.

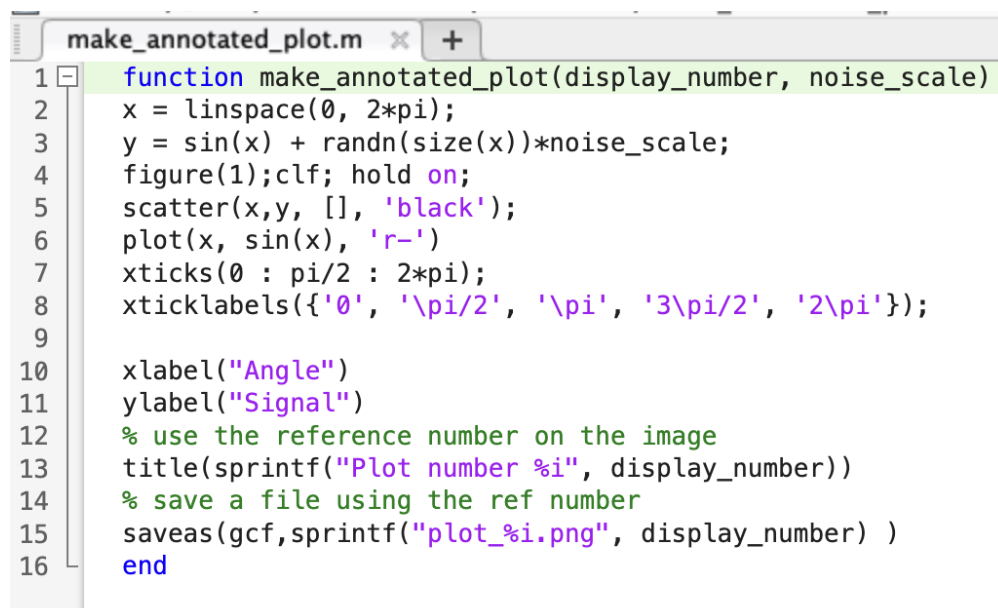
In this example I will create an ‘example’ directory in my home directory:



```
Host: login1.bouchet.ycrc.yale.edu Themes: Default
Last login: Tue Jun 30 16:34:04 2026 from 10.18.9.119
[akm76@login1.bouchet ~]$ pwd
/home/akm76
[akm76@login1.bouchet ~]$ mkdir example
[akm76@login1.bouchet ~]$ cd example/
[akm76@login1.bouchet example]$
```

When creating a cluster friendly code or script that you want to run, often you want to run the same code with different parameters, and possibly use a different variable as a job id. For my example, I’m going to plot a sine wave with a variable amount of noise, and then save what those plots look like.

My matlab code looks is below, you can upload that as a .m file using the ‘file’ utility on the ood homepage (see Figure 1), or write it out using a command line editor like vim or nano, or upload it using scp or rsync.



```
make_annotated_plot.m x +
1 function make_annotated_plot(display_number, noise_scale)
2 x = linspace(0, 2*pi);
3 y = sin(x) + randn(size(x))*noise_scale;
4 figure(1);clf; hold on;
5 scatter(x,y, [], 'black');
6 plot(x, sin(x), 'r-')
7 xticks(0 : pi/2 : 2*pi);
8 xticklabels({'0', '\pi/2', '\pi', '3\pi/2', '2\pi'});
9
10 xlabel("Angle")
11 ylabel("Signal")
12 % use the reference number on the image
13 title(sprintf("Plot number %i", display_number))
14 % save a file using the ref number
15 saveas(gcf,sprintf("plot_%i.png", display_number) )
16 end
```

Figure 2: matlab script

Now that you have the job script you can create the job file. One straightforward way is to copy the output into a txt file. A better way is to use the command line. You can do so by uploading the .py file to the example directory and running the following:

```

1  noise_parameters = [0.001, 0.01, 0.1, 1]
2
3  reference_number = 0
4
5  for noise_parameter in noise_parameters:
6      job_string = f"module load MATLAB; matlab -nodisplay -r \"make_annotated_plot({reference_number}, {noise_parameter});\""
7      reference_number = reference_number + 1
8      print(job_string)
9

```

Figure 3: Job creation script

```

[akm76@login1.bouchet example]$ python3 make_annotated_plot_jobs.py
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(0, 0.001);"
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(1, 0.01);"
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(2, 0.1);"
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(3, 1);"
[akm76@login1.bouchet example]$ python3 make_annotated_plot_jobs.py > example_job.txt
[akm76@login1.bouchet example]$ cat example_job.txt
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(0, 0.001);"
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(1, 0.01);"
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(2, 0.1);"
module load MATLAB; matlab -nodisplay -r "make_annotated_plot(3, 1);"
[akm76@login1.bouchet example]$

```

Figure 4: Job script output

The > syntax will redirect the output of the file to example_job.txt, you could also create the txt file on your local computer and upload it to the example directory, but I recommend doing as much as you can on the remote server.

You're almost ready to submit your job. To turn this list of commands into a job for the cluster I use dsq, a tool for doing just that. We need to load it using module, and then tell it where to find the job list, as well as the computational requirements of the job (more info here <https://docs.ycrc.yale.edu/clusters-at-yale/job-scheduling/dsq/>):

```

[akm76@login1.bouchet example]$ module load dsq
[akm76@login1.bouchet example]$ dsq --job-file example_job.txt --mem 10g -t 0-00:10:00 -p day
Batch script generated. To submit your jobs, run:
  sbatch dsq-example_job-2026-06-30.sh
[akm76@login1.bouchet example]$ sbatch dsq-example_job-2026-06-30.sh
Submitted batch job 16815574
[akm76@login1.bouchet example]$

```

Figure 5: Submit job to server queue

In this example, -job-file example_job.txt tells dsq where to find the jobs I want to run, -mem 10gb means each job is requesting 10gb of memory, the time requested is in the form D-HH:MM:SS, YCRC prefers that you do not submit many short jobs (this one is actually a pretty bad example) and instead merge short jobs into fewer longer jobs, aim for half an hour minimum. -p is the partition, the day partition is the most commonly used one and can run jobs for up to 1 day. More about partitions here docs.ycrc.yale.edu/clusters/bouchet/. There are more options with dsq and slurm that you can read about in their documentation.

dsq generates a .sh file using the information given to it, and then the sbatch command actually sends this job array to the scheduler in charge of the cluster.

To check on your job you can use squeue -u your_net_id or sacct -u your_net_id, try looking up the documentation for more info.

The expected output should be a .png containing a plot for each job with a distinct name, you can also check the output of the jobs in the .out files and some other info in the .tsv associated with the job. They will be saved by default in the example directory, visible here. And shown below.

Like uploads you can download files through the ood interface as well as scp or rsync.

```
[akm76@login1.bouchet example]$ squeue -u akm76
```

JOBID	PARTITION	NAME	USER	ACCOUNT	ST	SUBMIT TIME	TIME	TIME LIMIT	NODES	CPUS	MIN_MEMORY	TRES_PER_N	FEATURES	PRIORITY	NODELIST(REASON)
16843130_0-3	day	dsq-exam	akm76	pl_co54	PD	2026-06-30T22:35	0:00	10:00	1	1	10G	N/A	(null)	0.00000372 (Priority)	
16456316_12	pl_co54	dsq-msq	akm76	pl_co54	R	2026-06-26T22:37	3-23:58:01	4-00:00:00	1	4	50G	gres/gpu:1	(null)	0.00000384	a1118u16n01
16456316_13	pl_co54	dsq-msq	akm76	pl_co54	R	2026-06-26T22:37	3-23:58:01	4-00:00:00	1	4	50G	gres/gpu:1	(null)	0.00000384	a1118u16n01
16456316_14	pl_co54	dsq-msq	akm76	pl_co54	R	2026-06-26T22:37	3-23:58:01	4-00:00:00	1	4	50G	gres/gpu:1	(null)	0.00000384	a1118u18n01
16456316_15	pl_co54	dsq-msq	akm76	pl_co54	R	2026-06-26T22:37	3-23:58:01	4-00:00:00	1	4	50G	gres/gpu:1	(null)	0.00000384	a1118u18n01
16456316_16	pl_co54	dsq-msq	akm76	pl_co54	R	2026-06-26T22:37	3-23:58:01	4-00:00:00	1	4	50G	gres/gpu:1	(null)	0.00000384	a1118u18n01
16456316_17	pl_co54	dsq-msq	akm76	pl_co54	R	2026-06-26T22:37	3-23:58:01	4-00:00:00	1	4	50G	gres/gpu:1	(null)	0.00000384	a1118u20n01
16456316_19	pl_co54	dsq-msq	akm76	pl_co54	R	2026-06-26T22:37	3-23:58:01	4-00:00:00	1	4	50G	gres/gpu:1	(null)	0.00000384	a1118u20n01

Figure 6: squeue output, you can see the example job just submitted is waiting, I have some other jobs running that are also visible below it

```
[akm76@login1.bouchet example]$ ls
```

dsq-example_job-16843130_0-a1114u20n01.out	dsq-example_job-2026-06-30.sh	make_annotated_plot.m	plot_3.png
dsq-example_job-16843130_1-a1114u20n01.out	example_job.txt	plot_0.png	
dsq-example_job-16843130_2-a1114u20n01.out	job_16843130_status.tsv	plot_1.png	
dsq-example_job-16843130_3-a1114u20n01.out	make_annotated_plot_jobs.py	plot_2.png	

Figure 7: squeue output, you can see the example job just submitted is waiting, I have some other jobs running that are also visible below it

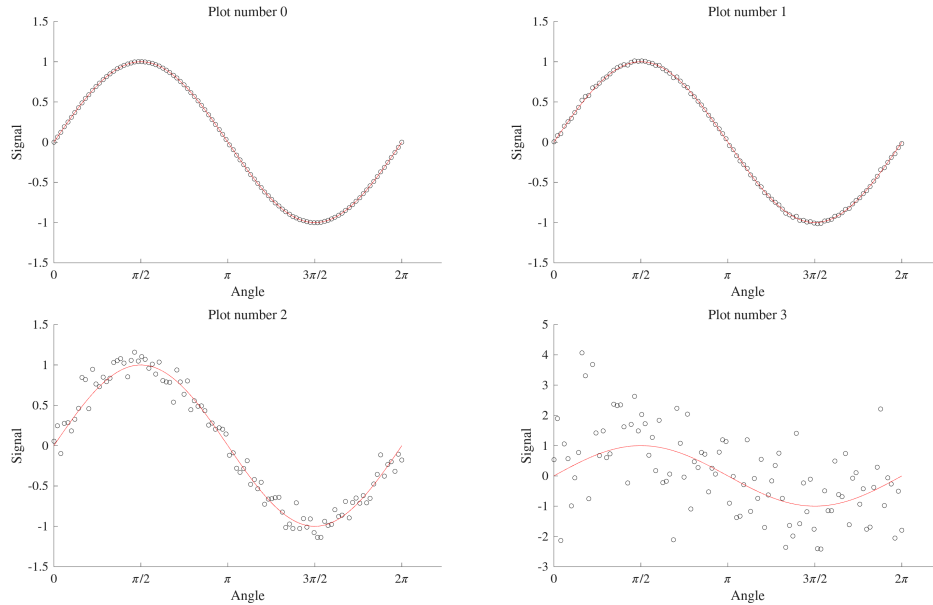


Figure 8: Figures with different levels of noise generated using this array job.